

Auditoria Completa

Grupo Amor Animal

@grupoamoranimal - Instagram para API

Gerado em: 19/08/2026 21:14

API: <https://api.projetosdinamicos.com.br/amoranimal>

Posts analisados: 360 | Registros API: 95

Período: 2024-10-30 a 2026-07-25

Documento gerado automaticamente por gerar-pdf.py

1. Resumo Executivo

Este documento apresenta a auditoria completa do projeto Grupo Amor Animal, que sincroniza postagens do perfil Instagram @grupoamoranimal com a API do site api.projetosdinamicos.com.br/amoranimal. O foco e explicar os processos, alternativas e complexidades de postar dados no site sem quebrar a integridade das tabelas.

Numeros Gerais

Metrica	Valor
Posts no Instagram	360
Periodo coberto	2024-10-30 a 2026-07-25
Registros na API	95
Payloads JSON prontos	109
Categorias mapeaveis	3 (adocao, procura-se, eventos)
Categorias sem tabela	2 (doacao, outros) - 63%% dos posts

2. Estado Atual da API

A API possui 6 tabelas. Estado atual:

/adocao: 2 registro(s)

- Nami
- Nami

/procura_se: vazio

/eventos: 3 registro(s)

- Feira de Adoção de Cães e Gatos -Cedvet
- 4º Mutirão de Castração de Gatos e Fei
- Feirinha de Adoção ? Amor que Transfor

/castracao: 89 registro(s)

- Gisleine Alves Pereira
- Daniela
- PAULO SERGIO DA SILVA

/adotado: vazio

/home: 1 registro(s)

- teste

Arquivos locais vs API

Os arquivos JSON da pasta (adocao.json, eventos.json, castracao.json, etc.) espelham exatamente o estado da API. Sao backups locais.

3. Pipeline de Dados (Instagram -> API)

3.1 Fluxo completo

```

    atualizar.js          Coleta posts do Instagram via imginn.com
    |
dataset.json           Classifica por categoria
    |
filtrar.js             Gera dataset-filtrado.json com payloads
    |
sincronizar.js         Envia payloads aprovados para a API
    |
sincronizados.json     Registra o que ja foi enviado (dedupe)
    |
API /adocao            Animais disponiveis
API /procura_se        Animais desaparecidos
API /eventos           Eventos e mutiroes
  
```

3.2 Scripts auxiliares

Script	Funcao
atualizar.js	Coleta posts do Instagram, classifica, gera dataset.js
filtrar.js	Organiza por categoria, gera payloads JSON por post
sincronizar.js	Envia payloads aprovados para a API (auth JWT)
simular-api.js	Simula tabelas da API sem enviar (preview)
cruzar-api.js	Cruza posts com tabelas, gera HTML de revisao
comparar-api.py	Compara dados Instagram x API
pagina-tabelas-api.py	Mostra tabelas da API com posts relacionados

4. Cadeia de Filtros (como posts sao pulados)

O sincronizar.js possui 4 camadas de filtro:

Camada 1 - Mapeamento de categorias

Apenas 3 categorias do Instagram tem tabela na API:

- adocao -> /adocao (105 posts)
- procura-se -> /procura_se (4 posts)
- castracao -> /eventos (23 posts, so com --castracao)

doacao (178) e outros (50) = IGNORADOS (228 posts, 63%%).

Camada 2 - Dedupe (sincronizados.json)

Arquivo sincronizados.json registra cada post ja enviado. Na proxima execucao, posts com code duplicado sao pulados. NAO EXISTE ainda (nenhum post sincronizado). Use --force para reenviar.

Camada 3 - Montagem do payload

O script extrai dados da legenda (nome, especie, porte, idade, foto). Se a extracao falhar, o post e pulado.

Camada 4 - Filtro de qualidade

Cada tabela tem um validador proprio:

/adocao - validarAdocao():

- Exige nome proprio real (nao "Voce", "Em", "Para")
- Nome: 2-12 caracteres, sem espacos
- Nao pode ser generico (STOP_NOME: 200+ palavras)
- Nao pode ser campanha/anuncio
- Nao pode ser pet ja adotado
- Legenda deve ter contexto de adocao

/procura_se - validarProcura():

- Exige nome proprio do pet
- Exige telefone de contato na legenda
- Legenda deve ter contexto de desaparecimento

/eventos - validarEvento():

- Data do evento identificada no texto
- Data deve ser FUTURA (passado = rejeitado)
- Legenda deve ter contexto de evento

5. Problemas Detectados

5.1 Payloads de adocao com qualidade ruim

Dos 105 posts de "adocao", a maioria NAO sao pets individuais - sao campanhas, feirinhas e avisos. O payload fica com nome = trecho da legenda.

```
POST: "Um dia pode mudar uma vida..."
Payload gerado:
nome: "Um dia pode mudar uma vida..." <- NOME ERRADO
especie: "felino" (detectado errado)
porte: null
foto_url: "https://scontent-..." (CDN expira!)
```

5.2 /castracao != posts do Instagram

A tabela /castracao (89 registros) e de FORMULARIOS de agendamento (ticket, clinica, agenda, cpf...). Os 23 posts de "castracao" do Instagram sao anuncios de mutiroes, nao formularios. O sincronizar.js nem inclui castracao por padrao.

5.3 Fotos com CDN temporal

URLs do CDN do Instagram expiram em horas. Sem imagem local em img-resumo-mensal/, o payload fica com URL quebrada.

5.4 /eventos so aceita eventos futuros

O filtro rejeita posts com data passada. Dos 3 eventos na API, 2 ja aconteceram.

5.5 63%% dos posts sem tabela

178 doacao + 50 outros = 228 posts sem correspondencia na API.

6. Alternativas para Filtrar Postagens

Para controlar quais posts são enviados ao site sem quebrar os dados, existem 7 alternativas práticas:

Alternativa 1 - Blacklist (lista de exclusão)

Complexidade: BAIXA | Escala: BOA | Risco: BAIXO

Criar `ignorar.json` com códigos que NUNCA devem ser enviados:

```
// ignorar.json
{
  "codes": ["DbGwiaU04n8", "Da6UtXiuEpS"],
  "obs": "posts de campanha"
}

// No sincronizar.js, após linha 358:
const bl = JSON.parse(fs.readFileSync("ignorar.json")).codes;
pendentes = pendentes.filter(p => !bl.includes(p.code));
```

Pro: Controle total. Contra: manual.

Alternativa 2 - Whitelist (enviar só estes)

Complexidade: BAIXA | Escala: RUIM | Risco: MINIMO

Listar apenas códigos que DEVEM ser enviados. Zero risco:

```
// enviar.json
{ "codes": ["DajSyzBFfMo", "DW62dpp08Yh"] }
```

Pro: Seguro. Contra: lista um por um.

Alternativa 3 - Qualidade mais rigorosa (auto)**Complexidade: MEDIA | Escala: ALTA | Risco: MEDIO**

Reforçar validadores com regras extras:

```
// Exigir foto local (base64)
if (!fotoBase64(p.code))
  return {ok:false, motivo:"sem foto local"};

// Rejeitar posts virais (>300 likes)
if (p.likes > 300)
  return {ok:false, motivo:"post viral"};

// Exigir indicador de pet disponivel
const IND = ["disponivel", "procura", "lar", "adotar"];
if (!IND.some(w => c.includes(w)))
  return {ok:false, motivo:"sem indicador"};
```

Pro: Automatico. Contra: pode rejeitar bons posts.

Alternativa 4 - Sincronizacao bidirecional**Complexidade: ALTA | Escala: ALTA | Risco: BAIXO**

Antes de enviar, consultar a API para ver se o pet ja existe. Se ja existe (mesmo nome), pular. Evita duplicatas sem depender de sincronizados.json.

```
// Buscar nomes ja na API
const apiAdocao = await fetch(API + "/adocao");
const listaApi = await apiAdocao.json();
const nomesApi = new Set(listaApi.map(r => norm(r.nome)));

// No loop: se ja existe, pular
if (nomesApi.has(norm(payload.nome))) {
  log("PULADO (ja existe) " + payload.nome);
  continue;
}
```

Pro: Evita duplicatas. Contra: matching por nome e impreciso.

Alternativa 5 - So posts recentes**Complexidade: BAIXA | Escala: BOA | Risco: MEDIO**

Sincronizar apenas posts dos ultimos N dias:

```
// No sincronizar.js:
const DIAS = 30;
const corte = new Date(Date.now() - DIAS*86400000).toISOString();
pendentes = pendentes.filter(p => p.data_iso > corte);
```

Pro: Simples. Contra: pula posts antigos uteis.

Alternativa 6 - Modo interativo**Complexidade: BAIXA | Escala: RUIM | Risco: MINIMO**

Perguntar post por post antes de enviar:

```

for (const p of pendentes) {
  console.log(p.code + " - " + p.titulo);
  const r = pergunta("Enviar? (s/n/q) ");
  if (r === "q") break;
  if (r !== "s") { pulado++; continue; }
  // enviar...
}

```

Pro: Controle humano total. Contra: lento.

Alternativa 7 - Status no dataset**Complexidade: MEDIA | Escala: ALTA | Risco: BAIXO**

Adicionar campo sync_status no dataset (pendente/aprovado/rejeitado):

```

// No atualizar.js:
function classificarPost(cap) {
  const {categoria, tags} = classificar(cap);
  let sync_status = "pendente";
  if (["doacao", "outros"].includes(categoria))
    sync_status = "nao_mapeado";
  return {categoria, tags, sync_status};
}

// No sincronizar.js:
pendentes = dataset.posts.filter(p =>
  mapRotas[p.categoria] &&
  p.sync_status !== "rejeitado" &&
  p.sync_status !== "nao_mapeado"
);

```

Pro: Integra com pipeline. Contra: altera estrutura do dataset.

7. Tabela Comparativa das Alternativas

Alt	Nome	Complexidade	Escala	Risco	Esforco
1	Blacklist	Baixa	Boa	Baixo	Baixo
2	Whitelist	Baixa	Ruim	Minimo	Medio
3	Qualidade rigorosa	Media	Alta	Medio	Medio
4	Bidirecional	Alta	Alta	Baixo	Alto
5	So recentes	Baixa	Boa	Medio	Baixo
6	Interativo	Baixa	Ruim	Minimo	Alto
7	Status dataset	Media	Alta	Baixo	Medio

Recomendacao

Combinar Alternativa 1 (blacklist) + Alternativa 3 (qualidade rigorosa). Controle manual para posts problematicos E filtro automatico para o resto.

8. Como Postar Corretamente

Método 1: Via sincronizar.js (automatico)

```
# 1. Atualizar dataset
node atualizar.js

# 2. Testar o que seria enviado
API_USER="user" API_PASS="pass" node sincronizar.js --dry-run

# 3. Enviar de verdade
API_USER="user" API_PASS="pass" node sincronizar.js
```

Método 2: Via curl (controle total)

```
# Autenticar
TOKEN=$(curl -s -X POST API_BASE/auth/login \
-H "Content-Type: application/json" \
-d '{"usuario":"USER","senha":"PASS"}' | \
python3 -c "import sys,json; print(json.load(sys.stdin)['token'])")

# POST /adocao (pet individual)
curl -X POST API_BASE/adocao \
-H "Content-Type: application/json" \
-H "Authorization: Bearer $TOKEN" \
-d '{"nome":"Nami","especie":"canino",
    "porte":"medio","status":"disponivel"}'
```

Regras de ouro

- So postar pets INDIVIDUAIS com nome proprio real
- Nome: "Nami", "Thor" ? NAO trecho de legenda
- Foto: base64 local ou URL permanente (nao CDN do Instagram)
- Eventos: so com data FUTURA
- Castracao: NAO postar do Instagram (tabela e para formularios)
- Doacao/outros: nao tem tabela, ficam so no Instagram
- Sempre usar --dry-run antes de enviar

9. Conclusao

O pipeline de sincronizacao Instagram -> API esta bem estruturado com atualizar.js, filtrar.js e sincronizar.js. Os filtros de qualidade rejeitam corretamente a maioria dos posts incorretos.

O principal desafio e que 63%% dos posts do Instagram nao tem correspondencia na API (doacao e outros). Dos posts mapeaveis, muitos sao campanhas/feiras disfarçados de adocao.

A recomendacao e usar blacklist + qualidade rigorosa, sempre validando com --dry-run antes de enviar. O arquivo sincronizados.json (dedupe) deve ser preservado apos cada execucao bem-sucedida para evitar duplicatas.

Proximos passos sugeridos

- Criar ignorar.json com os codes problematicos
- Rodar simular-api.js para ver simulacao antes de enviar
- Usar --dry-run sempre antes do envio real
- Baixar imagens locais via baixar-imagens.py para evitar CDN quebrado
- Considerar criar tabela /doacao na API para os 178 posts de doacao